

LABORATORY OBSERVATION

Course: Full Stack Web Development / Node.js

Course Code: 23CM4121

Observation Task: 5_2

Student Name: T. Akshaya

Roll No.: A24126552122

1. TITLE

Node.js File Management — File System Module Operations

2. AIM

To create a Node.js application that accepts a filename and content from the user and performs file creation, reading, appending, and re-reading operations using the File System (fs) module.

3. OBJECTIVE

- To create/write a file using `fs.writeFile()`.
 - To read file contents using `fs.readFile()`.
 - To append additional content using `fs.appendFile()`.
 - To take user input from the terminal using the `readline` module.
 - To display the final contents of the file after all operations.
-

4. THEORY

Node.js provides the built-in File System (fs) module to perform operations on files such as creating, reading, writing, appending, and deleting, all through asynchronous, callback-based functions so the program does not block while waiting for I/O.

The `readline` module lets a Node.js program take input from the user via the terminal, one line at a time, using `question()` calls.

In this program, `fs.writeFile()` creates a new file (or overwrites an existing one) with the given content, `fs.readFile()` reads it back as UTF-8 text, `fs.appendFile()` adds more content to the end of the file (prefixed with a newline), and the file is read once more to confirm the final state.

5. ALGORITHM / PROCEDURE

1. Import the fs and readline modules.
 2. Create a readline interface bound to process.stdin and process.stdout.
 3. Prompt the user for a filename.
 4. Prompt the user for initial content.
 5. Write the content to the file using fs.writeFile().
 6. Handle any error during writing; if none, confirm success.
 7. Read the file back with fs.readFile() and display the initial contents.
 8. Prompt the user for additional content.
 9. Append the additional content (with a leading newline) using fs.appendFile().
 10. Handle any error during appending; if none, confirm success.
 11. Read the file again and display the final contents.
 12. Close the readline interface.
-

6. TASK DESCRIPTION

Students learn how to work with the Node.js File System module, perform file operations, and develop basic server-side scripts.

A Node.js console application (task5_2.js) was built that:

- Accepts a filename and initial content from the user.
 - Creates the file and writes the content into it.
 - Reads and displays the initial file contents.
 - Accepts additional content and appends it to the file.
 - Reads and displays the final file contents after appending.
-

7. PROGRAM

`task5_2.js`

```
const fs = require("fs");
const readline = require("readline");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

rl.question("Enter filename: ", function (filename) {
  rl.question("Enter initial content: ", function (content) {
    fs.writeFile(filename, content, function (err) {
      if (err) {
        console.log("Error while creating file:", err);
      }
    });
  });
});
```


Line / Statement	Purpose
require("readline")	Loads the module for reading terminal input line by line.
readline.createInterface()	Creates an interface bound to stdin/stdout for prompting the user.
rl.question(...)	Displays a prompt and captures the user's typed response.
fs.writeFile(filename, content, cb)	Creates the file if it doesn't exist, or overwrites it if it does, with the given content.
fs.readFile(filename, "utf8", cb)	Reads the file's contents as text and passes them to the callback.
fs.appendFile(filename, "\n"+data, cb)	Adds a newline plus the new content to the end of the existing file, without erasing what's there.
err in each callback	Standard Node.js error-first callback pattern, checked before proceeding.
rl.close()	Closes the readline interface once all input/output work is done.

9. TEST CASES AND EXECUTION DATA

TC	Action	Expected Result	Actual Result	Status
TC-01	Enter filename student.txt and initial content 'Hello, this is my first line of content.'	File is created and initial content is displayed back	File created and content written successfully; initial content displayed exactly as entered	Pass
TC-02	Enter additional content 'This is additional appended content.' for the file from TC-01	Content is appended after the original text on a new line	Final file: original line + newline + appended line	Pass
TC-03	Press Enter with no text at the additional content prompt (empty string)	Program should not crash; an empty line is appended	'Content appended successfully.' printed; file ended with an extra blank line, no crash	Pass
TC-04	Enter an existing filename (student.txt) with new initial content 'Brand new overwritten content.'	fs.writeFile() should overwrite old data completely	Old content fully replaced; file contained only the new initial + appended content	Pass
TC-05	Enter a filename in a non-existent folder, e.g. nofolder/invalid_test.txt	fs.writeFile() should fail gracefully and print an error, not crash silently	Console printed ENOENT error with code, errno, syscall and path; program exited cleanly via rl.close()	Pass

10. OUTPUT

Output 1: TC-01 & TC-02 (create, read, append, final read)

```
PS C:\Users\syama\OneDrive\Desktop\122> cd observation_task
PS C:\Users\syama\OneDrive\Desktop\122\observation_task> node ./task5_2.js
Enter filename: student.txt
Enter initial content: Hello,this is my first line of content.
File created and content written successfully.

Initial file contents:
Hello,this is my first line of content.

Enter additional content: This is additional appended content.
Content appended successfully.

Final file contents:
Hello,this is my first line of content.
This is additional appended content.
```

Output 2: TC-03 (empty additional content)

```
PS C:\Users\syama\OneDrive\Desktop\122\observation_task> node ./task5_2.js
Enter filename: empty_test.txt
Enter initial content: Just the initial line.
File created and content written successfully.

Initial file contents:
Just the initial line.

Enter additional content:
Content appended successfully.

Final file contents:
Just the initial line.
```

Output 3: TC-04 (overwrite existing file)

```
PS C:\Users\syama\OneDrive\Desktop\122\observation_task> node ./task5_2.js
Enter filename: student.txt
Enter initial content: Brand new overwritten content.
File created and content written successfully.

Initial file contents:
Brand new overwritten content.

Enter additional content: Appended after overwrite.
Content appended successfully.

Final file contents:
Brand new overwritten content.
Appended after overwrite.
```

Output 4: TC-05 (invalid directory path, error handling)

```
PS C:\Users\syama\OneDrive\Desktop\122\observation_task> node ./task5_2.js
Enter filename: nofolder/invalid_test.txt
Enter initial content: This should fail.
Error while creating file: [Error: ENOENT: no such file or directory, open 'C:\Users\syama\OneDrive\Desktop\122\observation_task\nofolder\invalid_test.txt'] {
  errno: -4058,
  code: 'ENOENT',
  syscall: 'open',
  path: 'C:\\Users\\syama\\OneDrive\\Desktop\\122\\observation_task\\nofolder\\invalid_test.txt'
}
Focus folder in explorer (ctrl + click)
PS C:\Users\syama\OneDrive\Desktop\122\observation_task>
```

11. RESULT

Thus, a Node.js console application was successfully developed and tested using the File System (fs) and readline modules. The program correctly created, read, appended to, and re-read a file, and gracefully handled an invalid file path. All five test cases executed against the actual running program passed.